

I inspected **this exact uploaded ZIP** and ran the tests rather than relying on the previous review.

Bottom line

147/147 tests pass. The code is still NOT competition-ready.

The tests are mostly testing the implementation's own assumptions. They do not establish that the policy is economically correct or competitive against the real game.

More importantly, I found several concrete defects.

1. CRITICAL – the claimed premium-market protection is not implemented

`Scheduler._sell_orders()` says premium crops should be held when their market price has fallen below 50% of normal.

But the function explicitly does:

```
```python
current_stocks = {} # not available here; use 0 for price estimation
```
```

and then:

```
```python
live_price = MarketModel.market_price(rules, item, 0)
```
```

So it **always** evaluates the premium product at zero market stock.

For example, I tested MELON, WOOL, MILK and STRAWBERRY at their respective floor-reaching stock levels. The function still returned:

```
```text
['SELL', item, 5]
```
```

even though the actual market price at that stock is `$1`.

So the documented policy:

> "hold premium goods when the market has crashed"

is effectively **dead code**.

This is not a theoretical concern. It is a direct implementation bug.

****Severity: CRITICAL.****

2. CRITICAL – same-day planting/watering still does not satisfy the stated invariant

The code generates:

```
```text
PLANT priority = 6000
WATER same-day priority = 5999
```
```

and deliberately assigns the planting worker first.

That means with one worker:

```
```text
PLANT
```
```

is emitted, while:

```
```text
WATER
```
```

is left unassigned.

I reproduced this directly:

```
```text
tasks:
PLANT 6000
WATER 9500 same_day=True

actions:
farmer -> ['PLANT', 'MELON']
```
```

So the statement in the code that it guarantees same-day watering is false whenever there isn't a second available worker.

This is particularly important because the design document explicitly says a newly planted seed must receive same-day watering.

The test suite only tests ****existing dying crops****, not:

> newly planted crop + insufficient workforce → does the policy satisfy the planting-day invariant?

It doesn't.

****Severity: CRITICAL.****

3. CRITICAL – this is still not the component architecture you asked for

There are multiple Python files, but that is not enough.

The actual architecture is:

```
```text
main.py
 |
 +--> Scheduler
 |
 +--> MarketModel
 +--> Task generation
 +--> routing
 +--> Hungarian algorithm
 +--> animal logic
 +--> fertilizer logic
 +--> market orders
 +--> planting economics
 +--> worker action translation
...
```
```

`scheduler.py` is ****785 lines**** and contains a large fraction of the actual domain/strategy logic.

`main.py` is another 348-line stateful orchestration class.

So this is ****file-level modularization, not the component architecture you specified****.

The architecture should have had independently defined strategic components with explicit interfaces and ownership boundaries.

This is an architectural failure, independent of whether the tests pass.

4. HIGH – crop optimizer is still heuristic and contains invented economics

`MarketModel.crop\_values()` introduces:

```
```python
WATER_COST_PER_TURN = 2
STOCK_BUFFER_MULT = 2
```
```

Those are not derived from the actual engine.

It effectively assumes:

```
```text
opponent pressure $\approx 2 \times$ our expected production
```
```

and:

```
```text
watering cost $\approx$ $2 per worker-turn
```
```

Neither is established by the game.

This is particularly problematic because crop choice is one of the highest-leverage strategic decisions.

The code presents this as:

```
> "Improved model"
```

but it is actually a **hand-tuned heuristic**.

There is no replay-derived calibration behind those constants.

5. HIGH – the optimizer does not optimize against the actual terminal objective

The objective is terminal farm money.

The crop ranking instead maximizes approximately:

```
```text
(revenue - seed cost - estimated watering cost) / cycle length
```
```

That ignores important state interactions:

- * market price trajectory,
- * opponent production,
- * opponent selling,
- * worker travel,

- * shed capacity,
- * harvest congestion,
- * planting capacity,
- * cash availability,
- * crop timing relative to market demand,
- * terminal liquidation,
- * opportunity cost of worker actions.

Therefore:

****highest EV/turn $\neq$ highest terminal cash $\neq$ highest Bradley-Terry win probability.****

The code has no mechanism connecting crop selection to BT win probability.

6. HIGH – no Bradley-Terry robustness machinery exists

I searched the implementation.

There is no actual BT model.

No:

- * pairwise win matrix,
- * Bradley-Terry likelihood,
- * strength estimation,
- * confidence interval on strength,
- * paired-seed win-rate analysis,
- * opponent fixed effects,
- * seat effects,
- * chronological opponent robustness,
- * pool contamination detection.

Yet BT is the actual competitive objective.

The current `mc.py` measures ****terminal cash****, primarily against:

```
```python
OPPONENTS = {
 "starter": ...,
 "copy": ...
}
```
```

That's nowhere near the required competitive validation.

7. CRITICAL – the Monte Carlo harness can still produce misleading validation

Although the code improved exception accounting, the fundamental validation remains inadequate.

The available opponents are:

```
```text
PASS/starter
self-copy
```
```

So even a spectacular result against this pool tells us essentially nothing about the final BT tournament.

There is no:

```
```text
real replay agent A
real replay agent B
real replay agent C
...

```

pool.

No fresh-seed paired tournament.

No win/loss matrix.

No BT estimate.

Therefore a successful `mc.py` run **must not** be interpreted as competitive evidence\*\*.

---

## 8. HIGH – the "95% CI" is statistically weak

`mc.py` uses:

```
```python
1.96 * stdev / sqrt(n)
```
```

with `pstdev`.

That is a normal approximation to a mean, not a robust confidence interval for a small Monte Carlo sample.

More importantly, the samples aren't necessarily independent in the way this

calculation assumes, and **cash variance is not the relevant uncertainty for BT anyway**.

For BT validation we need uncertainty around **pairwise win probability / strength**, not merely terminal-money mean.

---

**## 9. HIGH – market order truncation can silently destroy intended strategy**

The scheduler can generate:

```
```text
SELL
HIRE
BUY_PRODUCT
```
```

orders exceeding the market-order limit.

``ActionEmitter`` simply does:

```
```python
self.market_orders[:self.max_market_orders]
```
```

That means later strategic orders silently disappear.

For example:

```
```text
SELL x many
HIRE x 10
BUY_PRODUCT FERTILIZER
```
```

can result in the fertilizer purchase never reaching the engine.

There is no feedback from the emitter telling the scheduler:

> "your order plan exceeded the available slots."

The policy therefore plans an action sequence that is different from what it actually submits.

---

**## 10. HIGH – fertilizer implementation is strategically disconnected**

The code has fertilizer logic:

```
```python
if yield_fertilized > yield_no_fertilizer:
```
```

but the actual policy is disabled:

```
```json
"FERTILIZER_ENABLED": false
```
```

So this is not a validated fertilizer strategy.

More importantly, the amount purchased is simply based on:

```
```text
number of beneficial standing crops - existing fertilizer
```
```

It doesn't establish whether fertilizer should be used on:

- \* wheat vs carrot,
- \* which crop,
- \* which growth stage,
- \* which worker,
- \* whether the additional yield will crash the market,
- \* whether the labor cost outweighs the incremental revenue.

So it is capability code, not an economically validated fertilizer policy.

---

## 11. HIGH – dead-reckoning worker state is still risky

When worker positions are absent, the system maintains its own:

```
```python
self.workers
```
```

and predicts movement from emitted actions.

That creates a second state machine.

If the engine rejects, modifies, delays, or otherwise interprets an action differently, the local state diverges.

The next turn can then route workers from **fictional positions**.

For a competition agent, the authoritative state should be the engine observation

whenever possible, and uncertainty should be handled explicitly rather than silently dead-reckoned.

---

## 12. HIGH – dead-reckoning harvest yield ignores fertilizer

This is concrete.

On:

```
```python
elif verb == "HARVEST":
```
```

the code adds:

```
```python
params.get("yield_no_fertilizer", 1)
```
```

regardless of whether the crop was fertilized.

So if fertilizer ever becomes enabled and produces an enhanced yield, the local worker state becomes wrong immediately.

That can then affect:

- \* carried load,
- \* DROP decisions,
- \* shed capacity calculations,
- \* future routing.

---

## 13. MEDIUM/HIGH – exception containment can hide a strategic catastrophe

The wrapper does:

```
```python
except Exception:
    ...
    return {
        "farmer": [],
        "hands": [],
        "market": []
    }
```
```

Operationally, protecting the season from a crash is sensible.

But this converts a potentially catastrophic implementation defect into **\*\*silent PASS behavior\*\*** during live play.

Telemetry records it, but the agent itself continues.

For validation, **\*\*one exception should be a release failure\*\***, regardless of whether the season technically survives.

---

**## 14. MEDIUM – `CareMonitor` claims sustained-idle evidence but doesn't implement a rolling idle window**

The documentation says:

> "only expands when the average idle fraction over the recent window exceeds the threshold"

But `note\_idle()` simply does:

```
```python
if idle_fraction > 0.1:
    self.capacity_target += 1
```
```

There is no idle history and no average.

So the implementation and its stated design are inconsistent.

This is another example of why passing unit tests isn't sufficient.

---

**# The biggest problem**

The code has become **\*\*better engineered around the heuristic design\*\***, but the heuristic design itself remains insufficient.

The repository has a lot of machinery now:

```
```text
RulesLoader
MarketModel
Forecaster
CareMonitor
Scheduler
Hungarian
ActionEmitter
Telemetry
```

```
MC
revalidation
compliance audit
```
```

That creates the appearance of a sophisticated system.

But the actual strategic loop is still essentially:

```
```text
observe state
  ↓
rank crops with heuristic EV
  ↓
generate chores
  ↓
prioritize chores
  ↓
assign workers
  ↓
sell inventory
  ↓
repeat
```
```

There is still no:

```
```text
real replay corpus
  ↓
opponent policy extraction
  ↓
strategy fingerprints
  ↓
candidate strategy generation
  ↓
paired fresh-seed matches
  ↓
win/loss matrix
  ↓
Bradley-Terry estimation
  ↓
robustness analysis
  ↓
strategy selection
```
```

And **that** is the layer that matters for winning the competition.

---

# Test result versus actual status

I ran the repository's tests:

**\*\*147 passed.\*\***

I also ran:

```
```text
revalidate.py
```
```

and got:

**\*\*5/5 acceptance checks passed.\*\***

But those results should **\*\*not\*\*** be reported as evidence that the agent is competition-ready.

They establish only that the code satisfies its current self-authored checks.

### My actual verdict

| Area                        | Verdict          |
|-----------------------------|------------------|
| -----                       | -----            |
| Python correctness          | ☑                |
| Existing unit tests         | ☑ 147/147        |
| Basic output contract       | ☑                |
| Basic routing               | ⚠                |
| Same-day planting invariant | <b>**X**</b>     |
| Premium market protection   | <b>**X**</b>     |
| Economic model              | <b>**X**</b>     |
| Replay-derived strategy     | <b>**X**</b>     |
| Real opponent pool          | <b>**X**</b>     |
| Bradley-Terry               | <b>**X**</b>     |
| BT robustness               | <b>**X**</b>     |
| Component architecture      | <b>**X**</b>     |
| Real-engine validation      | <b>**X**</b>     |
| Competitive evidence        | <b>**X**</b>     |
| Release                     | <b>**NO-GO**</b> |

**\*\*The most important correction is not another patch to `scheduler.py`.\*\*** The project needs to return to the architectural/validation gates: establish the real replay corpus, construct the exogenous opponent pool, implement the BT evaluation layer, and then build the strategy components around evidence from that evaluation.